Parcial Algoritmos y Programación II 7541 Curso Calvo (2° instancia) 22 de Junio 2009

Padrón: **Apellido y Nombre:**

Mail:

Parcialito de punteros: APROBADO - DESAPROBADO

1) Indicar la salida

```
/* Datos:
 *  '@' es 64
 *  'A' es 65
 */
int main(){

 int *A, *C, *F;
 int **B;
 char *D, *E;
 char G;
 int H;

 H = 65;
 G = (char) H;
 A = new int;
 (*A) = H;
 F = A;
 C = F;
 (*A) = 67;
 cout << (*A) << (*C) << (*F) << endl;

 B = &C;
 C = new int;
 (**B) = H + 1;
 cout << (*A) << (*C) << (**B) << endl;

 E = (char*) C;
 D = (char*) (*B);
 cout << (*E) << (*D) << endl;


 (*E) = G;
 cout << (*E) << (*C) << endl;

 F = (int*) D;
 while ( (*C) > 0) {
    (*C)--;
    cout << (*C) << (*D) << (**B) << endl;
    (*F) = (*F) - (**B);
 }

 delete A;
 delete C;

 return 0;
}
```

2) Un Blog está compuesto por un conjunto de Artículos. Cada Artículo posee un título, un texto y una lista de palabras clave. Todo Artículo puede tener asociado un conjunto de Comentarios. Cada Comentario tiene una descripción y una calificación. La calificación es un valor comprendido entre 1 y 10.
Se pide:
a) Indicar cuales son los TDA que intervienen en el planteo de esta situación , describiendo brevemente el objetivo de cada TDA.
b) Especificar la interfaz de las clases correspondientes a los TDA. Se debe indicar obligatoriamente pre y post condiciones de cada método, argumentos recibidos y tipo de retorno.
c) Implementar la clase Blog a partir de la siguiente especificación de su interfaz:

```
class Blog {

  public:
    /**
     * post: el Blog queda creado sin Artículos.
     */
    Blog();
    /**
     * post: destruye todos los Artículos que componen el Blog.
     */
    ~Blog();
    /**
     * post: devuelve los Artículos que componen el Blog.
     */
    Lista<Articulo*>* getArticulos();
    /**
     * post: devuelve una nueva Lista con los Artículos que tengan entre sus palabras clave palabraClave
     *       y que su calificación promedio (tomada de sus comentarios) esté comprendida entre
     *       calificadaDesde y calificadaHasta.
     */
    Lista<Articulo*>* buscarArticulos(string palabraClave, int calificadaDesde, int calificadaHasta);
}
```

3) Implementar una clase que modele el juego del Ahoracado. Un Ahorcado debe ser creada con la palabra a adivinar y la cantidad de errores permitidos. El Ahorcado debe poseer métodos para:

- Conocer la cantidad de letras que tiene la palabra.
- Conocer el estado del juego: JUGANDO, GANADO, PERDIDO.
- Proponer una letra, devolviendo la cantidad de apariciones que tiene esa letra en la palabra a adivinar. Si la letra no está en la palabra se incrementa la cantidad de errores cometidos. Sólo se pueden arriesgar letras mientra que está en estado JUGANDO. Cuando la cantidad de errores cometidos es mayor que la cantidad de errores permitidos, el juego queda PERDIDO.
- Conocer la cantidad de errores cometidos.
- Arriesgar una palabra, devolviendo si es correcta o no. Si es correcta el juego queda GANADO y en caso contrario PERDIDO.
- Conocer la palabra cuando el juego está GANADO o PERDIDO, si está JUGANDO debe devolver una cadena vacía.

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Mantener apagados celulares, i-pod, reproductores de mp3, etc. durante el examen
Duración del examen : 2 horas y 30 minutos